



Harness 잘 사용하기

챗봇과 싸우지 않고, 에이전트가 일할 환경을 설계하는 방법

게임플랫폼 1팀 라승수

시작하며

01

챗봇과 말씨름

맥락 충돌과 반복 설명

02

도구들의 수렵

자동완성·채팅 UI·터미널 에이전트

03

Harness

규칙·컨텍스트·권한·검증·상태 기록

개발자의 핵심 역량: AI에게 할 말 → AI가 안전하게 일할 환경

OPENING

전체 목차

01	코딩은 어디로 가고 있는가	개발자의 숙련 이동
02	왜 Claude Code인가	AI 코딩 도구의 구조 수렴
03	AI 시대의 개발 방법론	TDD·SDD·Spec-first 재부상
04	Harness	Prompt 이후의 에이전트 운영 구조
05	한계와 실패 패턴	긴 작업·보안·신뢰 문제
06	멀티 에이전트	역할과 컨텍스트 분리
07	워크플로우와 도구	명령어·게이트·워크트리·이슈
08	제작 과정	발표 제작 파이프라인 해부
09	다음에 해야 할 일	개인과 팀의 첫 변화

CHAPTER 01

코딩은 사라지는가

기계어 → 어셈블리

기계어

```
10111000 00000001 00000000 00000000 00000000
```

어셈블리

```
MOV EAX, 1
```



컴파일러가 만든 코드가 사람보다 효율적일 리 없다!

어셈블리 → C/Pascal

어셈블리

```
section .data
    msg db "Hello World"

section .text
    global _start

_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, 12
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```



C/Pascal

```
#include <stdio.h>

int main() {
    printf("Hello\n");
    return 0;
}
```

진짜 프로그래머는 파스칼 같은 걸 쓰지 않는다!

C → Java

C

```
#include <stdio.h>
#include <stdlib.h>

...

int main() {
    Point* p = (Point*)malloc(sizeof(Point));
    p->x = 10;
    p->y = 20;
    printf(
        "Point: %d, %d\n",
        p->x,
        p->y
    );
    free(p);
    return 0;
}
```



Java

```
Point p = new Point(10, 20);
System.out.println(
    "Point: " + p.x + ", " + p.y
);
p = null;
```

메모리를 직접 관리하지 않으면 진짜 개발자가 아니다!

Java → Python

Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```



Python

```
print("Hello, World!")
```

장난감 언어로 무슨 개발을 하냐!

AI 개발

이 함수를 리팩토링하고 테스트 코드를 작성해줘.

자연어로 시키는 건 진짜 개발이 아니다!

직접 하던 일의 감소

시대	전환	위임한 것
1950s	기계어 → 어셈블리	이진 코드를 직접 작성하는 부담
1960s	어셈블리 → C	레지스터와 명령어 최적화를 컴파일러에 위임
1990s	C/C++ → Java	메모리 해제를 가비지 컬렉터에 위임
2000s	언어 → 프레임워크	반복되는 보일러플레이트를 프레임워크에 위임
2010s	온프레미스 → 클라우드	서버 운영과 확장을 클라우드에 위임
2020s	직접 코딩 → AI 에이전트	코드 작성의 상당 부분과 작업 환경 설계

직접하는 일을 줄고, 설계하는 일은 늘어난다

숫자로 보는 변화

25%

YC W25

코드베이스 95% 이상 AI

100만 줄

OpenAI Codex 팀

수동 코드 한 줄 없음

3개월+

Meta 엔지니어

직접 코드 타이핑 없음

문서가 코드

이름은 다르지만 역할은 동일



Claude Code

CLAUDE.md



Cursor

Cursor Rules



GitHub Copilot

**copilot-
instructions.md**



Codex

AGENTS.md

개발자의 새로운 역할

코드를 잘 짜는 능력



문서를 잘 쓰는 능력



컨텍스트를 설계하는 능력

건축가

벽돌 쌓기 대신 설계도 작성

정확한 설계도 → 튼튼한 건물

오케스트라 지휘자

직접 연주 대신 전체 조율

CLAUDE.md = 총보

그래도 기초가 중요하다

코딩은 에이전트가 일할 환경을 설계하고 검증하는 영역으로 이동하고 있습니다.

변하는 것

직접 코드를 타이핑하는 비중

줄어든 것은 타이핑의 비중

변하지 않는 것

시스템을 이해하는 능력

안전성 · 유지보수성 · 기존 시스템 적합성

인증과 권한 · 동시성 · 데이터베이스 스키마 · 캐싱